CSCI 423
Introduction to Algorithms

DR. RAAFAT ELFOULY

- HW 1 issues

Useful Summation Formulas

$$\sum_{i=l}^{u} 1 = (1 + 1 + .. + 1) = u - l + 1$$

In Particular: $\sum_{i=1}^{n} 1 = n - 1 + 1 = n \in \theta(n)$

$$\sum_{i=1}^{n} i = 1 + 2 + .. + n = \frac{n(n+1)}{2} \approx \frac{n^2}{2} \in \theta(n^2)$$

$$\sum_{i=1}^{n} i^2 = 1^2 + 2^2 + .. + n^2 = \frac{n(n+1)(2n+1)}{6} \approx \frac{n^3}{3} \in \theta(n^3)$$

$$\sum_{i=0}^{n} a^i = a^0 + a^1 + .. + a^n = \frac{(a^{n+1} - 1)}{(a-1)} \text{ for any } a \neq 1$$

In Particular: $\sum_{i=0}^{n} 2^i = 2^0 + 2^1 + .. + 2^n = 2^{n+1} - 1 \in \theta(2^n)$

Recurrence Relations

- Can easily describe the runtime of recursive algorithms
- Can then be expressed in a closed form (not defined in terms of itself)

Recurrence Relation Form

- A recurrence relation is a recursive form of an equation, for example:

$$T(1) = 3$$

$$T(n) = T(n-1) + 2$$

- A recurrence relation can be put into an equivalent closed form without the recursion

Converting Recurrence Relations

The picture can't be displayed.

- Begin by looking at a series of equations with decreasing values of n:

$$T(n) = T(n-1) + 2$$
$$T(n-1) = T(n-2) + 2$$
$$T(n-2) = T(n-3) + 2$$
$$T(n-3) = T(n-4) + 2$$
$$T(n-4) = T(n-5) + 2$$

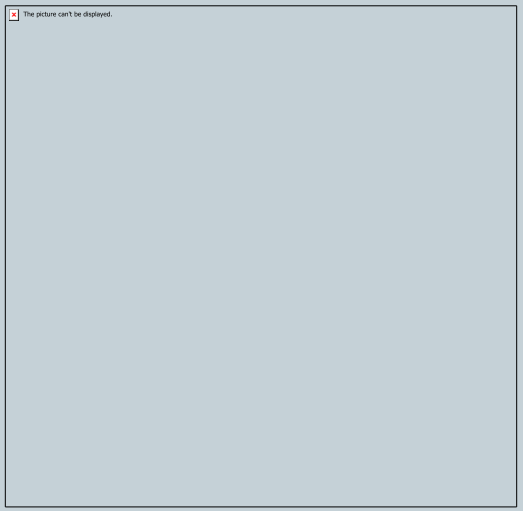
Converting Recurrence Relations

- Now, we substitute back into the first equation:

$$T(n) = T(n-1) + 2$$
$$T(n) = (T(n-2) + 2) + 2$$
$$T(n) = ((T(n-3) + 2) + 2) + 2$$
$$T(n) = (((T(n-4) + 2) + 2) + 2) + 2$$
$$T(n) = ((((T(n-5) + 2) + 2) + 2) + 2) + 2$$

i
1
2
3
4
5

Converting Recurrence Relations



- We stop when we get to $T(1)$:

$$\begin{aligned}
&\mathrm{T}(n) = \mathrm{T}(n-1) + 2 \\
&\mathrm{T}(n) = (\mathrm{T}(n-2) + 2) + 2 \\
&\vdots \\
&\mathrm{T}(n) = (\cdots((\mathrm{T}(1) + 2) + 2)\cdots + 2) + 2
\end{aligned}$$

- How many "+ 2" terms are there? Notice we increase them with each substitution.

Converting Recurrence Relations

The picture can't be displayed.

- We must have $n - 1$ of the "+ 2" terms because there was one at the start and we did $n - 2$ substitutions:

$$T(n) = T(1) + \sum_{i=1}^{n-1} 2$$

- So, the closed form of the equation is:

$$T(n) = 3 + 2(n-1)$$

Recurrence Relations

- Can easily describe the runtime of recursive algorithms
- Can then be expressed in a closed form (not defined in terms of itself)

- Consider the linear search:

EX. 1 - Linear Search

- Recursively
- Look at an element (constant work, c), then search the remaining elements…

- T(n) = T(n-1) + c
- “The cost of searching n elements is the cost of looking at 1 element, plus the cost of searching n-1 elements”

Linear Search (cont.)

- We'll "unwind" a few of these

$T(n) = T(n-1) + c$ (1)

But, $T(n-1) = T(n-2) + c$, from above

Substituting back in:

$T(n) = T(n-2) + c + c$

Gathering like terms

$T(n) = T(n-2) + 2c$ (2)

Linear Search (cont.)

- Keep going:

$$T(n) = T(n-2) + 2c$$

$$T(n-2) = T(n-3) + c$$

$$T(n) = T(n-3) + c + 2c$$

$$T(n) = T(n-3) + 3c \qquad (3)$$

- One more:

$$T(n) = T(n-4) + 4c \qquad (4)$$

Looking for Patterns

- Note, the intermediate results are enumerated
- We need to pull out patterns, to write a general expression for the k^{th} unwinding
  - This requires practise. It is a little bit art. The brain learns patterns, over time. Practise.
- Be careful while simplifying after substitution

EX. 1 – list of intermediates

Result at i^{th} unwinding	i
$T(n) = T(n-1) + 1c$	1
$T(n) = T(n-2) + 2c$	2
$T(n) = T(n-3) + 3c$	3
$T(n) = T(n-4) + 4c$	4

Linear Search (cont.)

- An expression for the kth unwinding:

  $T(n) = T(n-k) + kc$

- We have 2 variables, k and n, but we have a relation
- T(d) is constant (can be determined) for some constant d (we know the algorithm)
- Choose any convenient # to stop.

Linear Search (cont.)

- Let's decide to stop at T(0). When the list to search is empty, you're done…
- 0 is convenient, in this example…

  Let n-k = 0 => n=k
- Now, substitute n in everywhere for k:

  $T(n) = T(n-n) + nc$

  $T(n) = T(0) + nc = nc + c_0 = O(n)$

  (T(0) is some constant, c_0)

- For each of the followings:
  - Calculate T(1), T(2), T(3) , and T(4)
  - Derive closed form equation for T(n)
  - Check your answer by recalculating T(4)

- $T(n) = 2^* T(n-1) + 1 \quad n>1$
- $T(1)= 5$

Useful Summation Formulas

$$\sum_{i=l}^{u} 1 = (1 + 1 + .. + 1) = u - l + 1$$

In Particular: $\sum_{i=1}^{n} 1 = n - 1 + 1 = n \in \theta(n)$

$$\sum_{i=1}^{n} i = 1 + 2 + .. + n = \frac{n(n+1)}{2} \approx \frac{n^2}{2} \in \theta(n^2)$$

$$\sum_{i=1}^{n} i^2 = 1^2 + 2^2 + .. + n^2 = \frac{n(n+1)(2n+1)}{6} \approx \frac{n^3}{3} \in \theta(n^3)$$

$$\sum_{i=0}^{n} a^i = a^0 + a^1 + .. + a^n = \frac{(a^{n+1} - 1)}{(a-1)} \text{ for any } a \neq 1$$

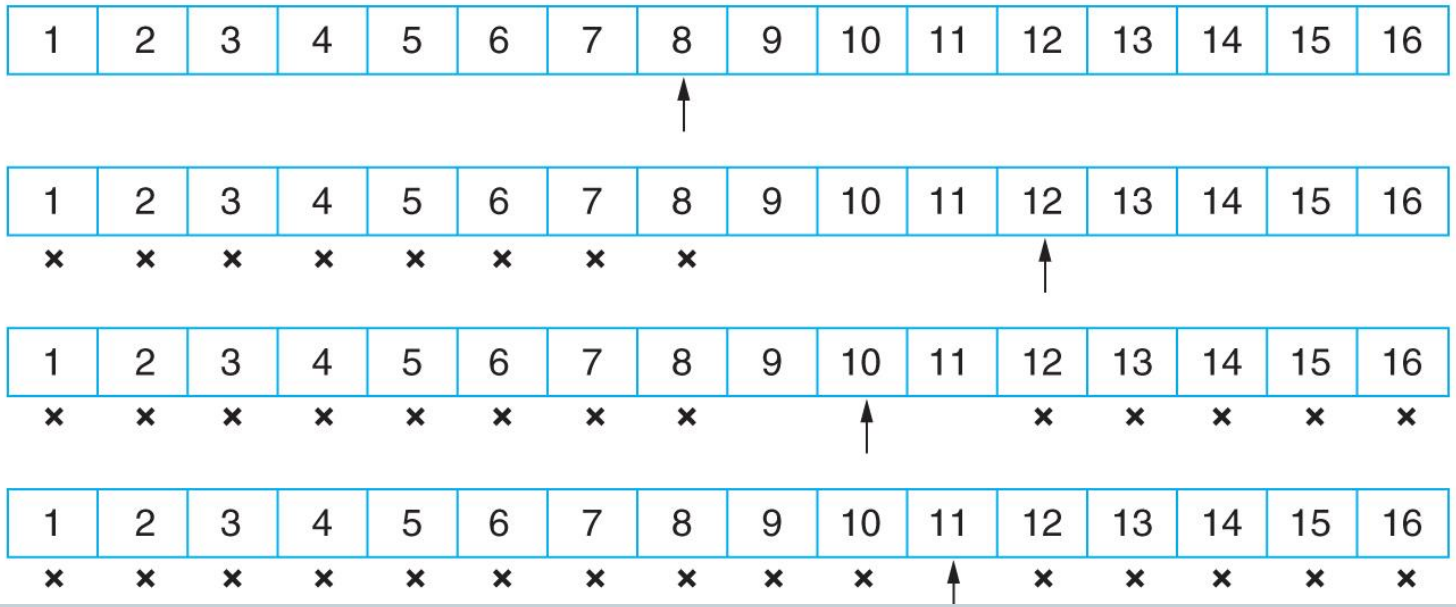
In Particular: $\sum_{i=0}^{n} 2^i = 2^0 + 2^1 + .. + 2^n = 2^{n+1} - 1 \in \theta(2^n)$

Binary Search

- Used with a sorted list
- First check the middle list element
- If the target matches the middle element, we are done
- If the target is less than the middle element, the key must be in the first half
- If the target is larger than the middle element, the key must be in the second half

Binary Search Example

FIGURE 3.2
Binary search



Binary Search Algorithm


```
start = 1
end = N
while start ≤ end do
  middle = (start + end) / 2
  switch (Compare(list[middle], target))
    case -1:  start = middle + 1
              break
    case  0:  return middle
              break
    case  1:  end = middle - 1
              break
  end select
end while
return 0
```

Binary Search

- Algorithm – “check middle, then search lower ½ or upper ½”
- T(n) = ???

Binary Search

- Algorithm – “check middle, then search lower ½ or upper ½”
- $T(n) = T(n/2) + c$

where c is some constant, the cost of checking the middle…

Binary Search (cont)

Let's do some quick substitutions:

$T(n) = T(n/2) + c$ (1)

but $T(n/2) = T(n/4) + c$, so

$T(n) = T(n/4) + c + c$

$T(n) = T(n/4) + 2c$ (2)

$T(n/4) = T(n/8) + c$

$T(n) = T(n/8) + c + 2c$

$T(n) = T(n/8) + 3c$ (3)

Binary Search (cont.)

Result at i^{th} unwinding	i
$T(n) = T(n/2) + c$	1
$T(n) = T(n/4) + 2c$	2
$T(n) = T(n/8) + 3c$	3
$T(n) = T(n/16) + 4c$	4

Binary Search (cont)

- We need to write an expression for the k^{th} unwinding (in n & k)
  - Must find patterns, changes, as i=1, 2, …, k
  - This can be the hard part
  - Do not get discouraged! Try something else…
  - We'll re-write those equations…
- We will then need to relate n and k

Binary Search (cont)

Result at i^{th} unwinding			i
T(n)	= T(n/2) + c	$=T(n/2^1) + 1c$	1
T(n)	= T(n/4) + 2c	$=T(n/2^2) + 2c$	2
T(n)	= T(n/8) + 3c	$=T(n/2^3) + 3c$	3
T(n)	= T(n/16) + 4c	$=T(n/2^4) + 4c$	4

Binary Search (cont)

- After k unwindings:

  $T(n) = T(n/2^k) + kc$

- Need a convenient place to stop unwinding – need to relate k & n

Binary Search (cont)

- After k unwindings:

  $T(n) = T(n/2^k) + kc$

- Need a convenient place to stop unwinding – need to relate k & n

- Let's consider $T(1) = c_0$
- So, let:

$$n/2^k = 1 \quad =>$$

$$n = 2^k \quad =>$$

$$k = \log_2 n = \lg n$$

Binary Search (cont.)

- Substituting back in (getting rid of k):

$$T(n) = T(1) + c \lg(n)$$

$$= c \lg(n) + c_o$$

$$= O(\lg(n))$$

Master Method

- BlackBox
- Divide problem into halves

The master method

The master method applies to recurrences of the form

$$T(n) = a\,T(n/b) + f(n)\ ,$$

where $a \geq 1$, $b > 1$, and f is asymptotically positive.

Approximating Recurrence Relations

The Master Method

If $T(n) \leq aT\left(\frac{n}{b}\right) + O(n^d)$

then

$$T(n) = \begin{cases} O(n^d \log n) & \text{if } a = b^d \quad \text{(Case 1)} \\ O(n^d) & \text{if } a < b^d \quad \text{(Case 2)} \\ O(n^{\log_b a}) & \text{if } a > b^d \quad \text{(Case 3)} \end{cases}$$

General Divide-and-Conquer Recurrence

$T(n) = aT(n/b) + f(n)$ where $f(n) \in O(n^d)$, $d \geq 0$

Master Theorem: If $a < b^d$, $T(n) \in O(n^d)$

If $a = b^d$, $T(n) \in O(n^d \log n)$

If $a > b^d$, $T(n) \in O(n^{\log_b a})$

Examples: $T(n) = 4T(n/2) + 1 \Rightarrow T(n) \in ?$

General Divide-and-Conquer Recurrence

$T(n) = aT(n/b) + f(n)$ where $f(n) \in O(n^d)$, $d \geq 0$

Master Theorem: If $a < b^d$, $T(n) \in O(n^d)$

If $a = b^d$, $T(n) \in O(n^d \log n)$

If $a > b^d$, $T(n) \in O(n^{\log_b a})$

Examples: $T(n) = 4T(n/2) + n \Rightarrow T(n) \in ?$

General Divide-and-Conquer Recurrence

$T(n) = aT(n/b) + f(n)$ where $f(n) \in O(n^d)$, $d \geq 0$

Master Theorem: If $a < b^d$, $T(n) \in O(n^d)$

If $a = b^d$, $T(n) \in O(n^d \log n)$

If $a > b^d$, $T(n) \in O(n^{\log_b a})$

Examples: $T(n) = 4T(n/2) + n^2 \Rightarrow T(n) \in ?$

General Divide-and-Conquer Recurrence

$T(n) = aT(n/b) + f(n)$ where $f(n) \in O(n^d)$, $d \geq 0$

Master Theorem: If $a < b^d$, $T(n) \in O(n^d)$

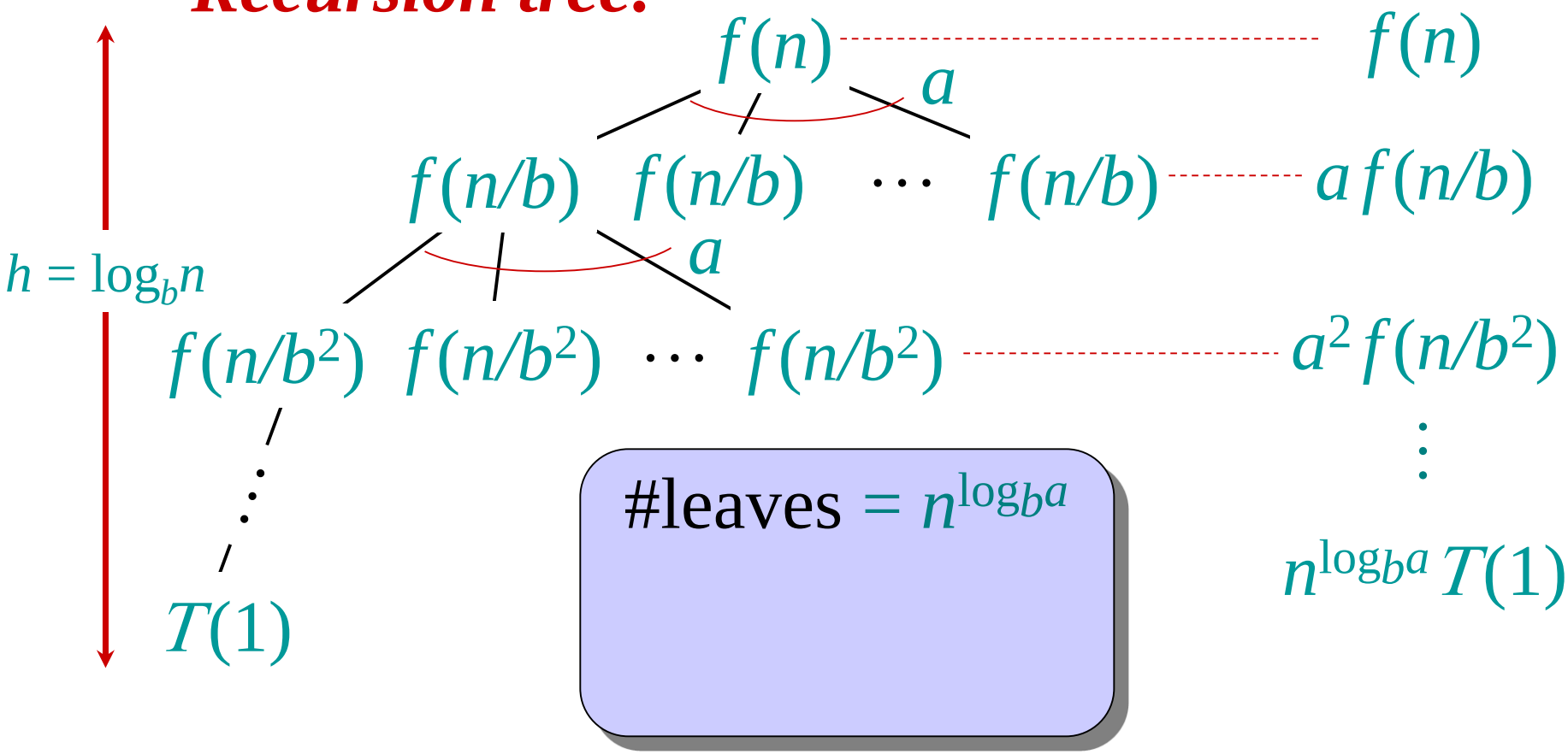
If $a = b^d$, $T(n) \in O(n^d \log n)$

If $a > b^d$, $T(n) \in O(n^{\log_b a})$

Examples: $T(n) = 4T(n/2) + n^3 \Rightarrow T(n) \in ?$

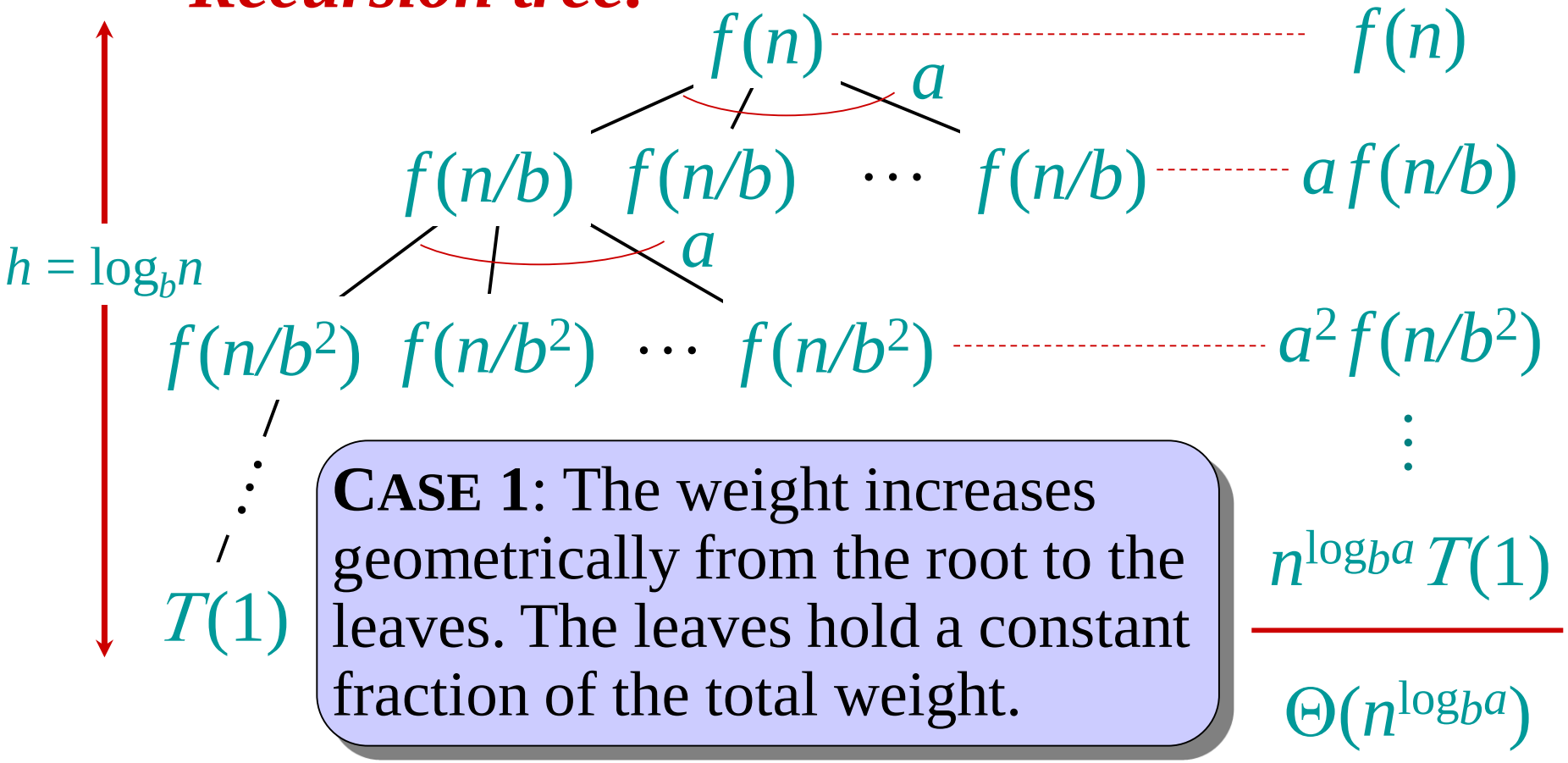
Idea of master theorem

Recursion tree:



Idea of master theorem

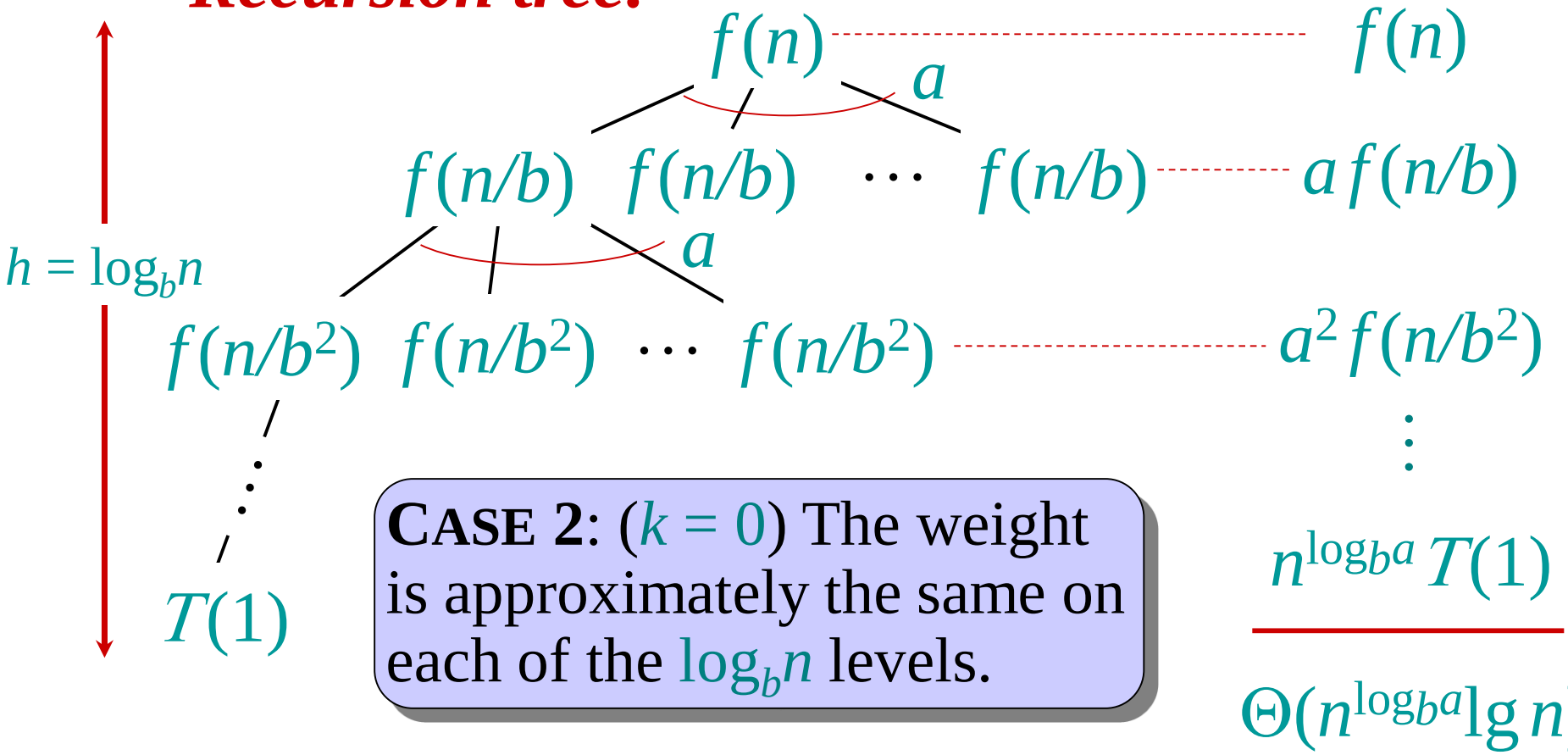
Recursion tree:



CASE 1: The weight increases geometrically from the root to the leaves. The leaves hold a constant fraction of the total weight.

Idea of master theorem

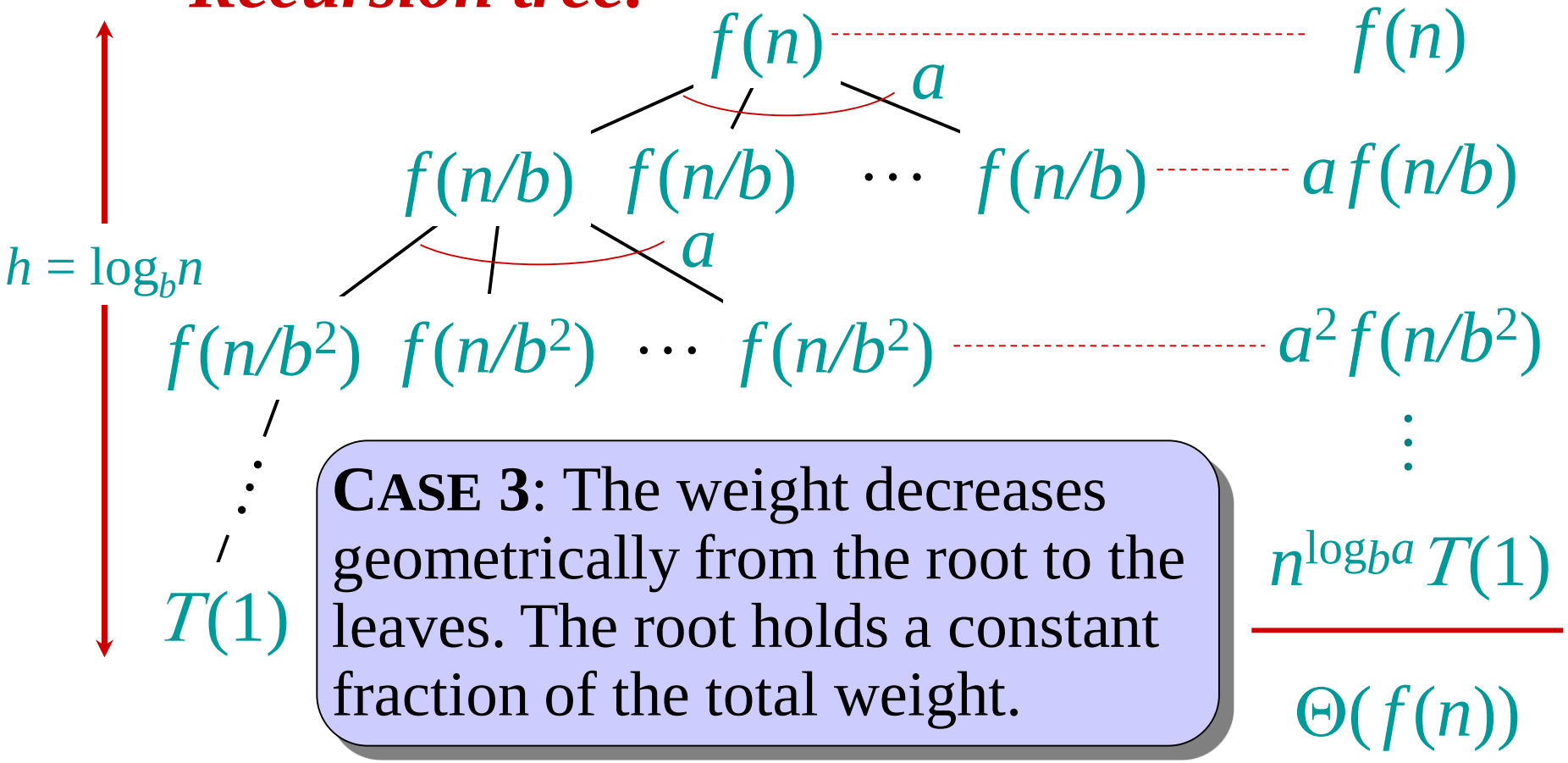
Recursion tree:



CASE 2: ($k = 0$) The weight is approximately the same on each of the $\log_b n$ levels.

Idea of master theorem

Recursion tree:



CASE 3: The weight decreases geometrically from the root to the leaves. The root holds a constant fraction of the total weight.

General Divide-and-Conquer Recurrence

$T(n) = aT(n/b) + f(n)$ where $f(n) \in O(n^d)$, $d \geq 0$

Master Theorem: If $a < b^d$, $T(n) \in O(n^d)$

If $a = b^d$, $T(n) \in O(n^d \log n)$

If $a > b^d$, $T(n) \in O(n^{\log_b a})$

Examples: $T(n) = 8T(n/4) + n \Rightarrow T(n) \in ?$

$T(n) = 2T(n/4) + n^2 \Rightarrow T(n) \in ?$

$T(n) = 4T(n/4) + n \Rightarrow T(n) \in ?$

Example 1

- Merge Sort
- a= 2
- b= 2
- d = 1
- case 2: O(nlogn)

$$T(N) = \begin{cases} \Theta\left(N^d\right) & \text{if } a < b^d \\ \Theta\left(N^d \lg N\right) & \text{if } a = b^d \\ \Theta\left(N^{\log_b a}\right) & \text{if } a > b^d \end{cases}$$

Binary search

Where are the respective values of a, b, d for a binary search of a sorted array, and which case of the Master Method does this correspond to? $T(n) = T(n/2) + c$

- Example 3:

- $T(n) <= 2T(n/2) + O(n^2)$

- a=2,
- b=2
- d=2
- $b^d = 4 > a$
- $T(n) = O(n^2)$

Closest Pair Problem

- Given a set of N points in space, which two are the closest?
- A brute force method calculates the distance between every pair of points using:

$$d(p_1, p_2) = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

- But this does $(N^2 - N)/2$ distance calculations

Closest-Pair Problem

Find the two closest points in a set of *n* points (in the two-dimensional Cartesian plane).

Brute-force algorithm

Compute the distance between every pair of distinct points

and return the indexes of the points for which the distance is the smallest.

Closest-Pair Brute-Force Algorithm (cont.)

ALGORITHM *BruteForceClosestPoints*(P)

```
//Input: A list P of n (n ≥ 2) points P_1 = (x_1, y_1), ..., P_n = (x_n, y_n)
//Output: Indices index1 and index2 of the closest pair of points
dmin ← ∞
for i ← 1 to n − 1 do
    for j ← i + 1 to n do
        d ← sqrt((x_i − x_j)^2 + (y_i − y_j)^2) //sqrt is the square root function
        if d < dmin
            dmin ← d; index1 ← i; index2 ← j
return index1, index2
```

Efficiency:

How to make it faster?

Brute-Force Strengths and Weaknesses

- Strengths
  - wide applicability
  - simplicity
  - yields reasonable algorithms for some important problems (e.g., matrix multiplication, sorting, searching, string matching)
- Weaknesses
  - rarely yields efficient algorithms
  - some brute-force algorithms are unacceptably slow
  - not as constructive as some other design techniques

Brute Force Algorithm


```
smallDist = ∞
  for i = 1 to N do
      for j = i+1 to N do
            dist = sqrt( (xi - xj)^2 + (yi - yj)^2)
            if dist < smallDist then
                  smallDist = dist
                  first = i
                  second = j
            end if
      end for
  end for
```

A Divide and Conquer Solution

- Divide the set of points in into a right and left half
- Find the closest pair in the right and left half (recursively)
- Determine (d_s) the shortest of these two distances
- Check if there is a pair of points within d_s units but on opposite sides of the dividing line that are closer

Divide and Conquer Example

FIGURE 2.1
A set of 12 points separated into two groups with six points in each part

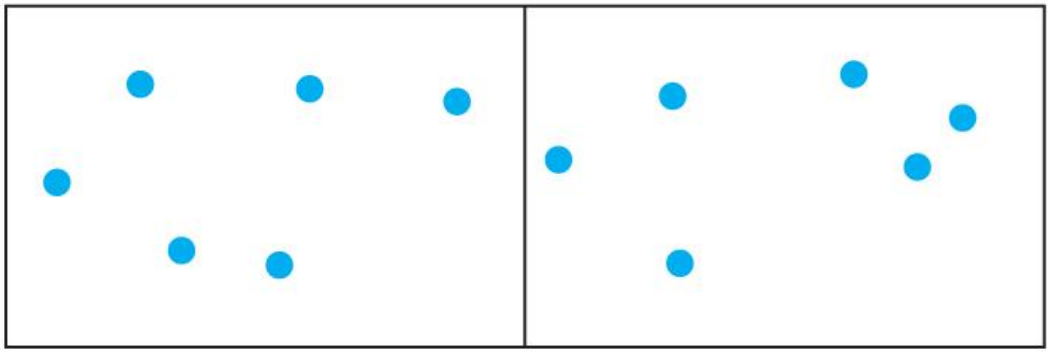
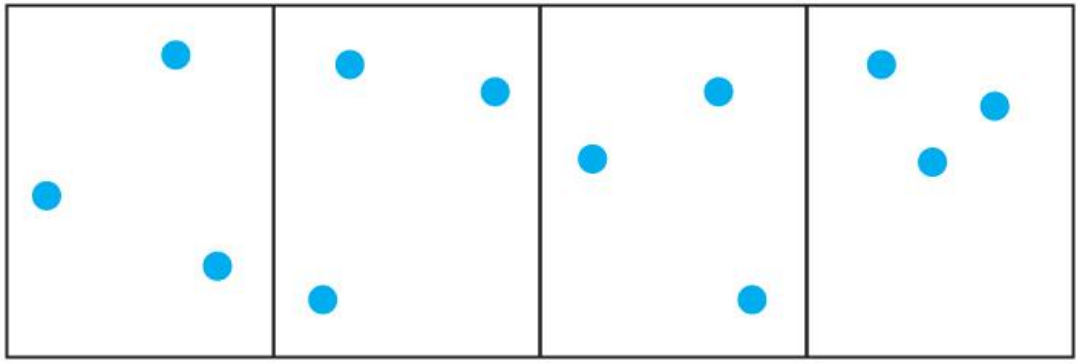


FIGURE 2.2
The left and right parts are subdivided again



Divide and Conquer Example

FIGURE 2.3
The closest points in each section are determined

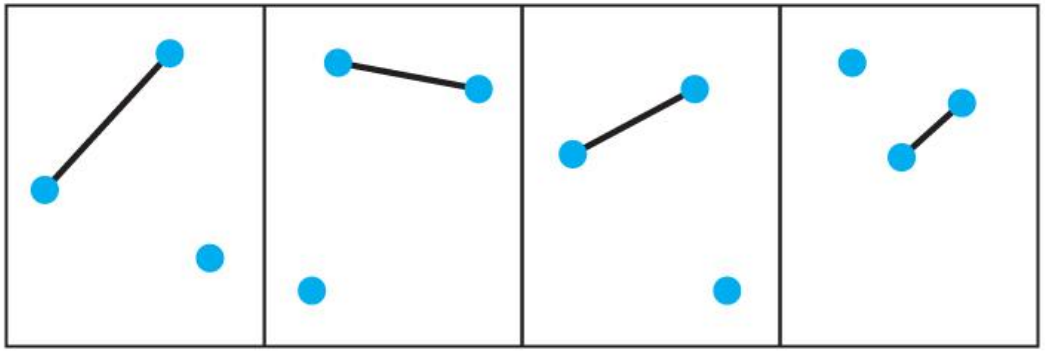
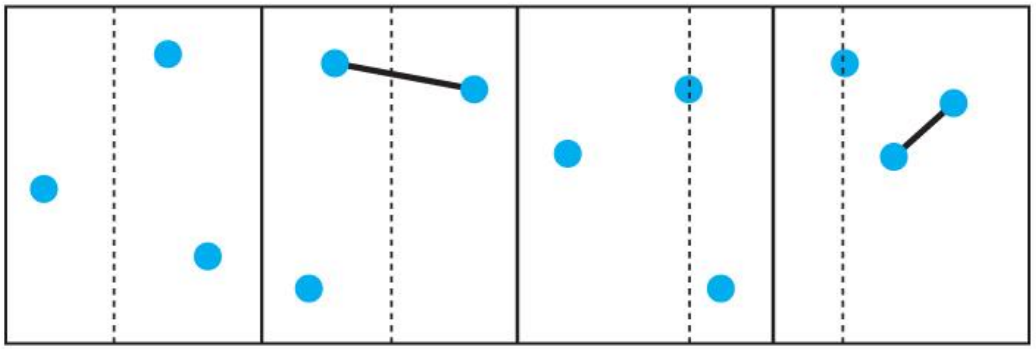


FIGURE 2.4
The area within d_s units of the subdivision is checked for a closer pair of points



Divide and Conquer Example

FIGURE 2.5
The closest pair of points in each half has been determined

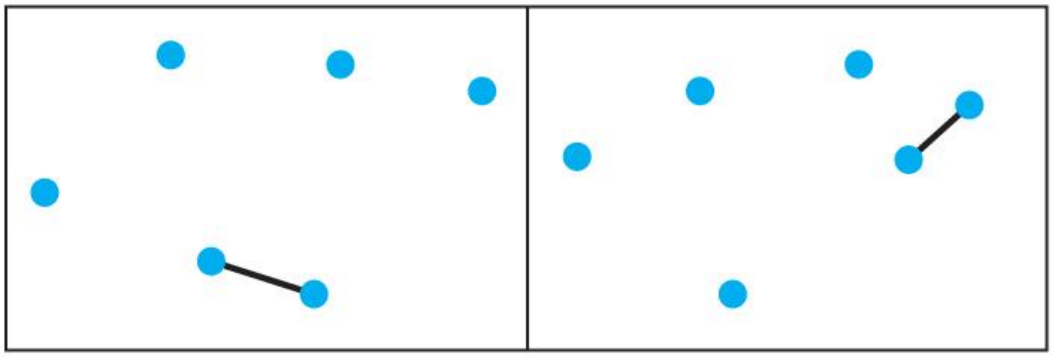
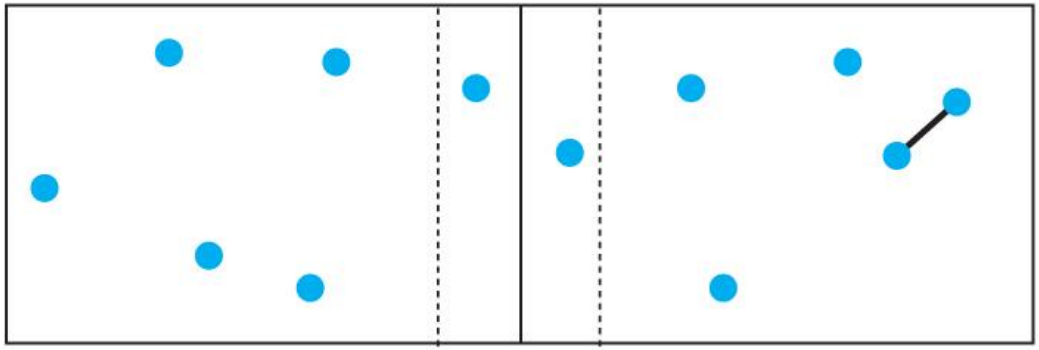
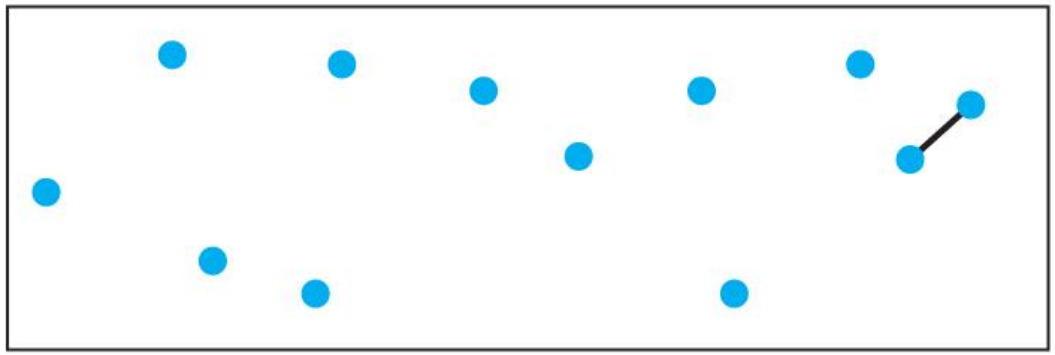


FIGURE 2.6
The area within d_s units of the center subdivision is checked for a closer pair of points



Divide and Conquer Example

FIGURE 2.7
The closest pair of points has been determined



Divide and Conquer Closest Pair Part 1


```
ClosestPair( Px, Py, d, p1, p2)
// Px   the points sorted by x coordinate
// Py   the points sorted by y coordinate
// d    the shortest distance between points p1 and p2
// p1   the first point
// p2   the second point

if sizeOf(Px) ≤ 3 then
  find d, p1, and p2 by brute force
  return
end if

construct Lx, Ly, Rx, Ry
ClosestPair(Lx, Ly, dl, L1, L2)
ClosestPair(Rx, Ry, dr, R1, R2)
```

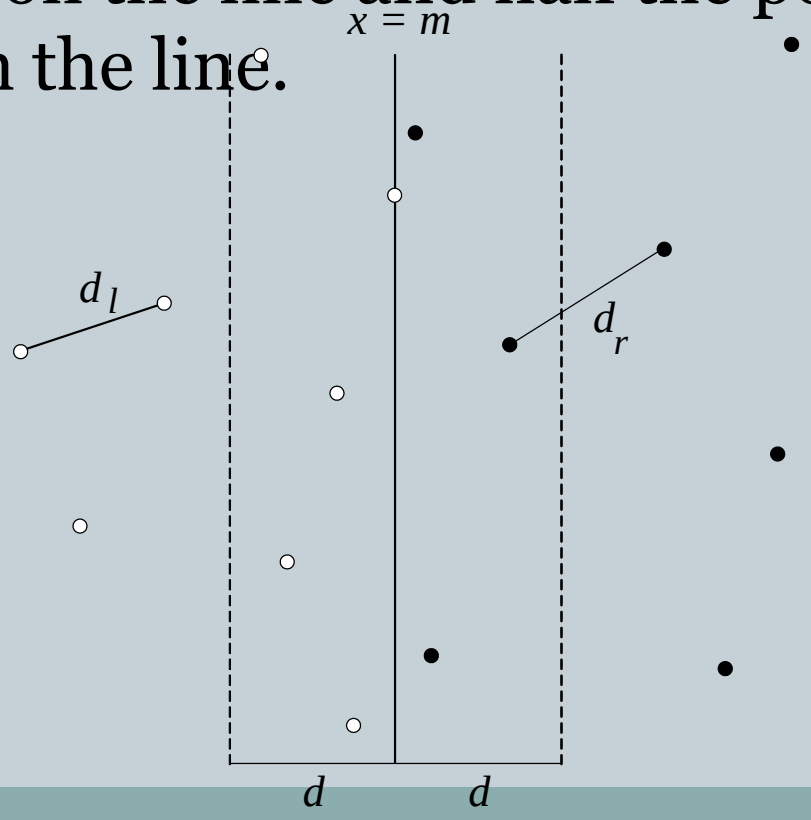
Divide and Conquer Closest Pair 2


```
d = minimum(dl, dr)
if d == dl then
   p1 = L1
   p2 = L2
else
   p1 = R1
   p2 = R2
end if

construct Mx and My
for i = 1 to sizeOf(My)-1 do
   for j = i+1 to i+7 (with j ≤ sizeOf(My)) do
         temp = distance(Myi, Myj)
         if temp < d then
                  d = temp
                  p1 = Myi
                  p2 = Myj
         end if
   end for
end for
```

Closest-Pair Problem by Divide-and-Conquer

Step 1 Divide the points given into two subsets P_l and P_r by a vertical line $x = m$ so that half the points lie to the left or on the line and half the points lie to the right or on the line.



Closest Pair by Divide-and-Conquer (cont.)

Step 2 Find recursively the closest pairs for the left and right subsets.

Step 3 Set $d = \min\{d_l, d_r\}$

We can limit our attention to the points in the symmetric vertical strip S of width $2d$ as possible closest pair. (The points are stored and processed in increasing order of their y coordinates.)

Step 4 Scan the points in the vertical strip S from the lowest up.

For every point $p(x,y)$ in the strip, inspect points in in the strip that may be closer to p than d. There can be no more than 5 such points following p on the strip list!

Efficiency of the Closest-Pair Algorithm

Running time of the algorithm is described by

$$T(n) = 2T(n/2) + M(n), \text{ where } M(n) \in O(n)$$

By the Master Theorem (with $a = 2$, $b = 2$, $d = 1$)

$$T(n) \in O(n \log n)$$

Divide and Conquer
Closest Point Analysis

- The combination step looks at the seven points closest to those points within d_s of the dividing line
- The points are divided into two halves
- Therefore, the recurrence relation is $T(N) = 2 * T(N/2) + \Theta(N)$
- This algorithm is $\Theta(N \lg N)$